

CMR Specialist

Complete Agent Manual - v2.1-shed: principles, mechanics, capabilities,
operator console, admin site, deployment, testing, troubleshooting

This manual describes the whole system as built and running: the controlled-agent core (model proposes, software enforces), the sealed tool module over a mock CMR database, the staff login system, the four-page observability admin site, the operator console, deployment with Postgres and backups, and the 41-check test suite. It is written so that an operator can work, an admin can inspect, and an engineer can extend.

1. Governing Principles

- **Store-first.** Every input is written to the database before any model is consulted. A crash mid-reasoning loses no user data.
- **Model proposes, software enforces.** The model contributes judgment (intent, names, plan order, drafts). The harness contributes guarantees (allowlists, step budget of 12, pattern checks, approvals, idempotency, audit).
- **Patterned IDs are never trusted from the model.** RRR, NIN, phone, email, plate, chassis must pass deterministic patterns; hallucinations are rejected, regex findings stand.
- **Names belong to the model.** Buyer/seller/requester names have no pattern; only the model extracts them, stored as *pending*, never *valid*.
- **Fallbacks are visible, never silent.** Model silence degrades to deterministic templates flagged *plan_fallback: true* and logged ([planner]/[extractor] lines).
- **Honest failure over invention.** No evidence means the bounded fallback; missing facts mean prompts; danger means the approval gate.
- **Two tool systems.** Internal tools read and compute (no audit rows). The sealed tool module is the sole CMR database writer (CHECK free, WRITE approval + audit).
- **Separation of work and internals.** Operators get the console; admins get a separate observability site. The two never mix.

2. Architecture at a Glance

Layer	Responsibility	Lives in
Ingress (web API)	Auth, guard, case/message/task/knowledge/admin endpoints, health, docs	harness/api/main.py, auth.py, admin.py
Agent intake	Intent (model-first + veto), extraction (model + regex floor), validation	harness/agent/intent, extractor, guard
Planning	Model orders steps from allowlist; harness enforces; plan stored at creation	harness/agent/planner, service.queue_task
Execution	Worker lease loop, 2-rule policy gate, idempotent step runner, drafting	harness/worker/, service.execute_task
Tool module (sealed)	CHECK/WRITE formulations; only holder of CMR access; overlay shares admin inserts	harness/tools/module, mockdb
Store	Cases, fields, tasks, steps, approvals, audit, knowledge, users, sessions	harness/store/ (14 tables)

2.1 Request lifecycle

POST /cases - guard - shell case row - input row - intent (model-first, rules veto) - extraction (model + regex merge, filler-word stripping) - branch: KNOWLEDGE_QUERY answers now; AUTOMATION_TASK creates a task **with its plan**

already stored (queued if complete, waiting_for_input with prompts if not); CASE_UPDATE attaches and resumes waiting tasks; UNSUPPORTED refuses with a fixed message.

2.2 The mock database (what it is and is not)

The mock CMR database is four in-memory Python dicts (profiles, vehicles, receipts, certificates) plus transfers/tokens, keyed by lookup key, with injectable faults (timeout, outage, not-found). It is **not relational**: no foreign keys, no joins, no reverse indexes (owner - vehicles is unanswerable). Its contract is behavioral: answer the six checks like the real registry would, fail like it would. The six-exam conformance suite is the cutover gate for a real connector.

2.3 Cross-process overlay

Web and worker are separate processes with separate mock copies. Admin inserts persist to a shared overlay file (var/mockdb_overlay.json) that every formulation fire merges additively - so worker runs see test rows without restarts, while in-process writes (transfers, tokens) are never clobbered. Reset clears the file and reseeds.

3. Intake, Planning, Execution (Mechanics)

3.1 Extraction merge rules

- Regex floor captures IDs and strips filler words ('plate number is ABC123XY' - 'ABC123XY').
- Model may ADD name fields freely; its patterned-ID values must pass the same patterns or are discarded; conflicts keep regex.
- Model silence keeps regex findings and logs *[extractor] model silent*.
- Names-only prompt rule: name fields hold just the name, no surrounding words.

3.2 propose_plan enforcement

Model replies with only JSON: ordered subset of the allowed tools, verifications first. Harness drops unknown names (a test once tried 'rm_rf_everything'), caps at 12 steps, force-marks WRITE steps requires_approval, assigns deterministic idempotency keys. Empty/silent/garbage proposals fall back to templates, logged and flagged.

3.3 Worker, policy gate, idempotency, resume

- Worker leases queued tasks (status running + leased_by); queue age and heartbeat on /health expose stalls.
- Policy: (1) approval - flagged steps or approval_required != never park the task in waiting_approval; one human approval unlocks, never loops. (2) confinement - unregistered tools fail the task with error_ref.
- Steps run in order, args/results/status appended to run_steps; formulation steps route via thin client; completed idempotency keys are skipped on replay; leases expire back to queue up to 3 attempts.
- Follow-ups (POST /cases/{id}/messages) merge facts (versioned) and auto-resume waiting tasks whose requirements are now met.

4. Knowledge Base (Procedure Memory)

Upload stores the file and chunks text; Publish flips one version to *active* (archiving siblings). Retrieval searches ACTIVE versions only - upload-without-publish is the top cause of empty answers, by design. Answers draft from cited spans through a validator: no spans means the bounded fallback. The mock DB is fact memory; the knowledge base is procedure memory.

4.1 Bulk upload (admin site)

Knowledge page: multi-file picker, per-file staging with chunk counts (*uploaded X of Y*), then publish-all with a second counter (*published X of Y*). Staging and publishing stay distinct - the counter shows both, so ungrounded answers are always traceable to an unpublished version.

5. Login System (The Locks)

- **Identity**: username + PBKDF2-hashed password per staff member (stdlib only, 210k rounds). Cleartext never stored.
- **Session**: 12-hour bearer token (the hand stamp) kept in browser storage, sent as Authorization header; logout revokes it.

- **Bootstrap:** the first-ever user is open creation and becomes admin; afterwards creation requires authentication, and granting admin requires an admin.
- **Migration:** pre-admin databases gain users.is_admin automatically; the oldest user is promoted when no admin exists.
- **Actor everywhere:** approvals record the decider's username (session preferred over any supplied name); machine callers keep working via the legacy API key as actor 'api-key'.
- **Admin gate:** is_admin flag guards the whole observability API (401 anonymous, 403 non-admin) and the admin site.

6. Capabilities and Task Lifecycle

6.1 Task families

- **payment_reconciliation** (RRR + date + account): validate - lookup - verify receipt - draft. Straight-through when complete.
- **change_of_ownership** (plate + buyer + seller): model names, regex plate, validate - lookup - draft. Waits with prompts when incomplete.
- **general_support / knowledge Q&A:** cited spans or the bounded fallback.
- **Approval-gated work:** approval sensitive/always or WRITE-bearing plans pause for a named human decision, recorded with reason.

6.2 States and visibility flags

State	Meaning	Move it by
queued	Complete facts; awaiting worker	automatic (seconds)
waiting_for_input	Missing facts; plan already visible	follow-up with the missing fact
waiting_approval	Sensitive/WRITE steps need a human	Approval card / admin decision
running / completed / failed	Executing / done / error kept	poll; error_ref explains

- **plan_proposed_by:** *model* (reasoned) or *template* (fallback cover) - on intake responses, task payloads, console badge.
- **plan_fallback: true** means work continued without model reasoning. Clusters mean throttling/quota - check logs, not code.

7. Operator Console Guide (<http://localhost:8080>)

Card	Action	Result
1 - New case	Paste question/task, Submit	intent, task_id, status, missing + prompts, plan flags
2 - Follow-up	Case ID carries; paste fact, Attach	merge; waiting tasks resume (resumed=N)
3 - Queue task	Type + approval level, Queue	direct task; approval:always demos the gate
4 - Knowledge	Upload, then Publish version id	chunks; unpublished = unsearchable
Case state	Refresh case / Refresh task	fields, plan + plan:model/template(fallback) badge
Approval	Approve / Reject + reason	gated tasks proceed or stop, recorded with your name

7.1 Exercises

- A - wait then resume. Card 1: *Musa sold his car to Adaeze Okafor, please transfer the ownership* (waiting, missing plate). Card 2: *the plate number is ABC123XY* (resume - completed).
- B - straight-through. Card 1: *Please reconcile my Remita payment RRR 123456789012 made yesterday for account acct-X* (queued - completed, paid, cited spans).
- C - approval gate. Card 3, approval:always, payment_reconciliation, *confirm receipt 123456789012* (waiting_approval; clear in Approval card).
- D - unhappy path. Card 1 with RRR 000000000000 (completed-with-not-found, never invented).

8. Admin Site Guide (<http://localhost:8080/admin>)

Separate multi-page site, same API, admin-gated. Sign in on Home; the badge flips to ADMIN. Deliberately unlinked from the operator console - the separation is the safety feature.

8.1 The four pages

Page	Does	Guardrail
MockDB explorer	Browse seed tables with counts; insert test rows; reset to seed	TEST banner; rows tagged synthetic + author + time; mock adapter only - can never reach a real registry
Harness inspector	Recent cases with task badges; full task detail by ID	Read-only: no state change possible; all mutations stay in the API
Worker monitor	Heartbeat, queue distribution, oldest-queued age, 10 recent tasks, 5s auto-refresh	Read-only; replaces log-file watching
Bulk knowledge upload	Multi-file picker; uploaded-X-of-Y then published-X-of-Y counters	Staging distinct from publishing; ungrounded answers trace to unpublished versions

8.2 Linked test-data template (profile - car - certificate - receipt)

Insert in order; keys must match across tables (owner name, plate, account):

```
profiles: {"phone": "08099998888", "email": "testdriver@example.com",
"profile_id": "prof-900", "username_type": "phone", "name": "Test Driver"}
vehicles: {"plate": "XYZ999AB", "chassis": "CHS000111222",
"vehicle_id": "veh-900", "owner": "Test Driver", "owner_history": 1}
certificates: {"plate": "XYZ999AB", "status": "approved",
"request_age_h": 5, "cert_no": "CMR-0900"}
receipts: {"rrr": "999900001111", "status": "paid", "amount": 5000,
"linked_account": "testdriver@example.com"}
```

Note: one profile may own several vehicles (same owner name on multiple plates), but ownership is a label, not a link - 'all vehicles for this owner' is unanswerable in the mock and is specified as future connector work.

9. API, Data, Configuration

9.1 Endpoint essentials

Method + path	Purpose	Notes
POST /cases	Create case + intake	text (+title/sender); returns plan flags
POST /cases/{id}/messages	Follow-up	merges facts; resumes waiting tasks
POST /cases/{id}/tasks	Queue task directly	type + instructions + approval level
GET /cases/{id} - GET /tasks/{tid}	State + detail	plan, flags, steps, approvals+approver
POST /tasks/{tid}/approvals	Human decision	approved/rejected + reason; actor recorded
POST /knowledge/documents + .../publish	Stage + activate	both steps required
POST /auth/users - login - logout; GET /auth/me	Locks	bootstrap, session, identity
GET /admin/mockdb, /queue, /cases/recent; POST /admin/mockdb/...	Observability	admin-gated; inserts synthetic
GET /health - /docs	Liveness + API docs	watch oldest_queued_job_age_s

9.2 Data model (14 tables)

cases, case_inputs, extracted_fields (versioned, pending vs valid), tasks (plan_json), agent_runs, run_steps (args/result/status), approvals (requester/approver/reason), tool_audit (append-only; 0 rows means nothing mutated the CMR DB), knowledge_documents/versions/chunks, citations, case_events, users, user_sessions.

9.3 Configuration

Key	Purpose	Notes
INFERENCE_PROVIDER / OPENROUTER_* / GEMINI_*	Model backend	OpenRouter needs model + Bearer + Referer/Title
INFERENCE_API_KEY	Model billing	flash-class cost is cents/day
HARNESS_API_KEY / TOOL_MODULE_TOKEN	API + module auth	rotate per environment
DATABASE_URL	Store	Postgres for pilot; SQLite fallback in var/
HARNESS_DB_PATH / MOCKDB_OVERLAY	File locations	default under var/ (gitignored)
LLM_MIN_INTERVAL_S / CACHE_TTL_S	Pacer + cache (2s/600s)	paces, never blocks
POSTGRES_PASSWORD / BACKUP_KEEP_DAYS	Compose secrets + retention	14-day backup pruning

Config never auto-loads .env: web starts with --env-file .env, worker with exported variables. Forgetting this is the top local-deploy fault.

10. Deployment (The Shed)

- **Machine:** one always-on office PC/mini-PC/VM (2 CPU/4GB plenty); web + worker auto-start on boot.
- **Lock the door:** Caddy/HTTPS front; one login per staff member; approvals carry names.
- **Safe:** Postgres via compose (SQLite fallback in var/); nightly pg_dump service with 14-day pruning; 90-day auto-purge (python -m harness.store.purge) that unlinks audit rows from purged cases.
- **Paper:** one-page data rules (what is stored, pilot-only eyes, 90-day life, sign-off) + one-page desk sheet (three statuses, golden rules, support number).
- **Rollout:** one team, payments first (read-only), ownership second, WRITE-gated work only after the approval habit exists; two shadow weeks (system proposes, staff verify).

10.1 Operations

```
# start (project root; .env required)

nohup .venv/bin/python -m uvicorn harness.api.main:app \
--host 0.0.0.0 --port 8080 --env-file .env > /tmp/cmr_server.log 2>&1 &

nohup env $(grep -v '^#' .env | grep '=' | xargs) \
.venv/bin/python -m harness.worker.worker > /tmp/cmr_worker.log 2>&1 &

# health / stop

curl -s http://127.0.0.1:8080/health

kill -f harness.api.main; pkill -f harness.worker.worker
```

Slow first starts (up to ~2 min on a loaded machine) are normal. Empty curl right after launch usually raced startup - retry. Port busy means a stale process is squatting :8080 - kill by PID first.

11. Verification and Troubleshooting

11.1 Test suite (41 checks, must stay green)

File	Proves	N
test_lean_spec.py	Spec lifecycle: intake, waiting, approvals, idempotency	7
test_tool_module.py	Catalog, pipeline, audit, faults, conformance gate	13
test_llm_gemini.py	Provider wiring, cache, veto, merge	12
test_model_plans.py	Model ordering; invented tools dropped; silent fallback	3
test_shed_auth.py	Bootstrap, login, password refusal, actor on approvals	3
test_admin_site.py	Admin gating, synthetic insert + reset, page serving	3

Note: the suite shares one test database (engine singleton) - auth helpers register-then-login so files pass in any order.

11.2 Failure signatures

- Stuck *queued* + growing queue age: worker down - restart; check heartbeat (admin Worker page).
- Perpetual *waiting_for_input*: Refresh case names the missing fact - answer via Card 2.
- *plan: template (fallback)* clusters: model throttled - server log [planner]/[extractor] lines; wait, retry.
- Knowledge answers empty: version uploaded but not published - publish it (admin counters show staged vs active).
- 401 on admin pages: sign in on Home; 403: account is not admin.
- Synthetic row not found by worker: check overlay file exists (var/); reset clears it.
- Buyer/seller with extra words: names-only prompt guards this; report exact text for hardening.

11.3 Repository rules (standing)

- Branches are workspaces (shed/..., admin/..., tidy/...); versions are tags (v2.0, v2.1-shed). main always boots.
- .env and var/ never committed. Commits small and readable. Suite green before merge.